DEX Aggregator for Uniswap V2 and V3

A UUPS-Upgradeable On-Chain Router with Mathematical Guarantees

BLOCKCHAIN2 Course, Project 1

April 2026

Abstract

We present an on-chain decentralized exchange (DEX) aggregator for Uniswap V2 and V3 deployed on the Ethereum Sepolia testnet. The protocol enumerates all admissible single-hop routes across the two automated market-maker (AMM) families — four Uniswap V3 fee tiers and the Uniswap V2 constant-product pool — selects the one maximizing output, and executes the swap atomically with slippage and deadline protection. A subsequent UUPS upgrade extends the routing set with multi-hop paths through a configurable list of intermediate tokens. We provide a rigorous analysis of the underlying AMM invariants, prove monotonicity, boundedness, and concavity of the V2 swap curve, derive the single-hop optimality of the aggregator, and characterize the regime in which multi-hop strictly dominates the direct route. All contracts are verified on Sourcify with exact bytecode match, and the protocol passes 38 Foundry tests including unit, property-based (fuzz), and Sepolia fork tests against live liquidity.

Contents

1 Introduction **2**

2 Preliminaries and Notation **2**

3 Analysis of the Uniswap V2 Swap Curve **3**
3.1 Derivation of the swap formula . . . 3
3.2 Monotonicity . . . 3
3.3 Boundedness . . . 4
3.4 Concavity and diminishing returns . . . 4

4 Uniswap V3 in a Single Active Tick Range **5**

5 The Aggregator Protocol **5**
5.1 Single-hop aggregation . . . 5
5.2 Multi-hop extension . . . 6

6 Implementation **7**
6.1 Architecture . . . 7
6.2 Storage layout (ERC-7201) . . . 7
6.3 Security model . . . 8
6.4 Upgrade authorization . . . 8

7 Deployment and Evaluation **8**
7.1 Sepolia deployment . . . 8
7.2 Live swap demonstration . . . 8
7.3 Testing . . . 9

8 Conclusion 9

1 Introduction

Automated market makers (AMMs) have become the dominant liquidity provisioning mechanism in decentralized finance. Each AMM exposes a pricing curve that deterministically maps an input amount to an output amount, parameterized by reserves and a fee. Uniswap V2 [1] implements the *constant-product* rule, while Uniswap V3 [2] generalizes to *concentrated liquidity* with four standard fee tiers: 0.01 %, 0.05 %, 0.30 %, and 1.00 %.

For any given token pair, the effective price offered by V2 and the four V3 tiers may differ. A user swapping naïvely through one venue leaves value on the table whenever a competing venue would have returned more output for the same input. An *aggregator* closes this gap by quoting across all venues and routing each swap through the venue that strictly dominates at the moment of execution.

Contributions.

1. A UUPS-upgradeable Solidity implementation (`DexAggregatorV1`) that aggregates Uniswap V2 and all four V3 fee tiers in a single on-chain call (Section 6).
2. A rigorous mathematical analysis of the V2 swap curve, including monotonicity, boundedness, and strict concavity (Section 3, Theorems 3.3–3.7).
3. An analysis of V3 within a single active tick range (Section 4).
4. An optimality proof for the single-hop aggregator (Theorem 5.2) and a characterization of the multi-hop advantage (Proposition 5.4).
5. A Sepolia deployment and end-to-end swap demonstration, with all contracts verified on Sourcify (Section 7).

Scope. This paper assumes AMM semantics *in isolation*: we do not model front-running or sandwich attacks (slippage bounds mitigate these operationally but do not eliminate them), and we treat Uniswap V3 as a family of concentrated-liquidity pools without modeling tick crossings within a single trade. Under these assumptions the aggregator's optimality claim reduces to a finite maximum over venue outputs and is therefore immediate (Theorem 5.2). The non-trivial content of the paper is in the analysis of individual venues and in the multi-hop characterization.

2 Preliminaries and Notation

Throughout the paper we fix an ordered token pair $(\mathsf{In}, \mathsf{Out})$ and work with non-negative real reserves. All quantities are expressed in the smallest indivisible units of each token (for example, wei for ETH-denominated tokens); integer truncation in the on-chain implementation is discussed in Remark 3.2.

Definition 2.1 (V2 pool state)**.** A *Uniswap V2 pool state* is a triple (x, y, γ) with

$$x \in \mathbb{R}_{>0}, \quad y \in \mathbb{R}_{>0}, \quad \gamma \in (0, 1],$$

where x is the reserve of the input token, y the reserve of the output token, and γ the proportion of input that is credited against the invariant (for canonical Uniswap V2, $\gamma = 997/1000 = 0.997$ corresponds to the 0.3 % fee).

Definition 2.2 (V2 swap function)**.** Given a V2 pool state (x, y, γ), the *swap function* $\phi_{x,y,\gamma} : \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}$ is

$$\phi_{x,y,\gamma}(\delta) \;=\; \frac{y\,\gamma\,\delta}{x + \gamma\,\delta}, \qquad \delta \in \mathbb{R}_{\geq 0}. \tag{1}$$

Definition 2.3 (V3 pool state, single active tick range)**.** Within a single active tick range $[P_a, P_b] \subset \mathbb{R}_{>0}$, a Uniswap V3 pool is characterized by a *liquidity* $L \in \mathbb{R}_{>0}$ and a current price $P_c \in [P_a, P_b]$. We denote the triple (L, P_a, P_b) and assume for the analysis of Section 4 that the trade is small enough not to cross P_a or P_b.

Definition 2.4 (Venue set)**.** For a fixed token pair, the *venue set* available to the aggregator is

$$\mathcal{V} \;=\; \{v_0\} \cup \{v_{100}, v_{500}, v_{3000}, v_{10000}\},$$

where v_0 denotes the V2 pool (if it exists) and v_f denotes the V3 pool at fee tier f basis points. A venue is *live* if the corresponding pool exists on-chain and has non-zero liquidity; otherwise its output is taken to be 0.

3 Analysis of the Uniswap V2 Swap Curve

We derive (1) from the constant-product invariant and prove three properties of $\phi_{x,y,\gamma}$ that justify its use as an AMM pricing rule.

3.1 Derivation of the swap formula

Proposition 3.1 (V2 invariant)**.** *Let (x, y) be the pre-trade reserves of a V2 pool and let $\delta \geq 0$ be the input amount. Suppose the output amount $\Delta \in [0, y)$ satisfies the invariant*

$$(x + \gamma\delta)(y - \Delta) \;=\; x\,y. \tag{2}$$

Then $\Delta = \phi_{x,y,\gamma}(\delta)$.

Proof. Solve (2) for Δ:

$$y - \Delta = \frac{xy}{x + \gamma\delta} \iff \Delta = y - \frac{xy}{x + \gamma\delta} \;=\; \frac{y(x + \gamma\delta) - xy}{x + \gamma\delta} \;=\; \frac{y\,\gamma\,\delta}{x + \gamma\delta}.$$

The formula matches (1), and $\Delta < y$ is verified in Theorem 3.5 below. □

Remark 3.2 (Integer implementation)**.** The Solidity implementation uses the equivalent integer form

$$\phi^{\text{int}}_{x,y,997}(\delta) \;=\; \left\lfloor \frac{997\,\delta \cdot y}{1000\,x + 997\,\delta} \right\rfloor, \tag{3}$$

which corresponds to (1) with $\gamma = 997/1000$ and applies integer truncation. Truncation is favorable to the pool (it can only decrease Δ), which preserves the no-arbitrage property of the invariant.

3.2 Monotonicity

Theorem 3.3 (Strict monotonicity)**.** *For every fixed pool state (x, y, γ), the function $\phi_{x,y,\gamma} : \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}$ is C^∞ on $\mathbb{R}_{\geq 0}$ and strictly increasing on $(0, \infty)$.*

Proof. Since $x > 0$ and $\gamma > 0$, the denominator $x + \gamma\delta > 0$ for all $\delta \geq 0$, so ϕ is smooth on $\mathbb{R}_{\geq 0}$. Differentiating:

$$\begin{aligned}\phi'_{x,y,\gamma}(\delta) &= \frac{(y\gamma)(x+\gamma\delta) - (y\gamma\delta)(\gamma)}{(x+\gamma\delta)^2} \\ &= \frac{y\gamma x + y\gamma^2\delta - y\gamma^2\delta}{(x+\gamma\delta)^2} \\ &= \frac{y\gamma x}{(x+\gamma\delta)^2}.\end{aligned}$$

Since $x, y, \gamma > 0$ and the denominator is positive, $\phi'_{x,y,\gamma}(\delta) > 0$ for all $\delta \geq 0$, which establishes strict monotonicity. □

Corollary 3.4 (Marginal output at zero input)**.** *The* marginal rate at zero *satisfies*

$$\phi'_{x,y,\gamma}(0) \;=\; \frac{y\,\gamma}{x}.$$

This is the AMM-equivalent of the spot price y/x, scaled by the surviving fraction of the input γ.

3.3 Boundedness

Theorem 3.5 (Output bound)**.** *For every pool state (x, y, γ) and every $\delta \in \mathbb{R}_{\geq 0}$,*

$$0 \;\leq\; \phi_{x,y,\gamma}(\delta) \;<\; y.$$

Moreover, $\lim_{\delta\to\infty} \phi_{x,y,\gamma}(\delta) = y$.

Proof. The lower bound is immediate since numerator and denominator of (1) are both non-negative. For the upper bound, rewrite

$$\phi_{x,y,\gamma}(\delta) \;=\; y \cdot \frac{\gamma\delta}{x+\gamma\delta} \;=\; y \cdot \left(1 - \frac{x}{x+\gamma\delta}\right).$$

Since $x > 0$, we have $x/(x+\gamma\delta) > 0$ for all $\delta \in \mathbb{R}_{\geq 0}$, so $\phi_{x,y,\gamma}(\delta) < y$. As $\delta \to \infty$, $x/(x+\gamma\delta) \to 0$, so $\phi_{x,y,\gamma}(\delta) \to y$. □

Remark 3.6. Theorem 3.5 reflects the fundamental *no-draining* property of constant-product AMMs: no finite amount of input can drain the output reserve. This property is asserted as a fuzz invariant in `DexAggregatorFuzz.t.sol` (`testFuzz_CalcV2AmountOut_NeverExceedsReserve`).

3.4 Concavity and diminishing returns

Theorem 3.7 (Strict concavity)**.** *For every fixed pool state (x, y, γ), the function $\phi_{x,y,\gamma}$ is strictly concave on $\mathbb{R}_{\geq 0}$; that is, $\phi''_{x,y,\gamma}(\delta) < 0$ for all $\delta \geq 0$.*

Proof. From the proof of Theorem 3.3, $\phi'_{x,y,\gamma}(\delta) = y\gamma x/(x+\gamma\delta)^2$. Differentiating once more:

$$\phi''_{x,y,\gamma}(\delta) \;=\; y\gamma x \cdot \frac{d}{d\delta}\left[(x+\gamma\delta)^{-2}\right] \;=\; y\gamma x \cdot \left(-2\,(x+\gamma\delta)^{-3} \cdot \gamma\right) \;=\; -\,\frac{2\,y\,\gamma^2\,x}{(x+\gamma\delta)^3}.$$

All factors y, γ, x are positive, and the denominator is positive, so $\phi''_{x,y,\gamma}(\delta) < 0$ for every $\delta \in \mathbb{R}_{\geq 0}$. □

Corollary 3.8 (Marginal rate is strictly decreasing)**.** *The marginal rate $\phi'_{x,y,\gamma}$ is strictly decreasing on $\mathbb{R}_{\geq 0}$. Equivalently, the effective* price impact *$\pi(\delta) := \delta/\phi_{x,y,\gamma}(\delta)$ is strictly increasing in δ on $(0, \infty)$.*

Proof. Strict decrease of ϕ' is equivalent to strict concavity of ϕ (Theorem 3.7). For the price-impact statement, compute

$$\pi(\delta) \;=\; \frac{\delta}{\phi_{x,y,\gamma}(\delta)} \;=\; \frac{\delta\,(x+\gamma\delta)}{y\gamma\delta} \;=\; \frac{x}{y\gamma} + \frac{\delta}{y}.$$

This is linear in δ with positive slope $1/y$, hence strictly increasing. □

Remark 3.9 (Economic reading)**.** Corollary 3.8 expresses *diminishing returns to volume*: larger trades obtain a strictly worse effective rate. This is the source of price impact and the reason the aggregator benefits from splitting large trades (Proposition 5.4).

4 Uniswap V3 in a Single Active Tick Range

Uniswap V3 parameterizes liquidity by tick ranges, allowing liquidity providers to concentrate capital around a target price. A full analysis requires tracking tick crossings, virtual reserves, and tick-level accounting; see [2] for a complete treatment. For a trade that does not cross any active tick boundary, V3 reduces to a V2-like constant-product computation on *virtual reserves*.

Proposition 4.1 (Virtual-reserve equivalence within a tick range)**.** *Let (L, P_a, P_b, P_c) characterize an active V3 tick range with liquidity L, price bounds $[P_a, P_b]$, and current price $P_c \in [P_a, P_b]$. Define the* virtual reserves

$$x_v \;=\; \frac{L}{\sqrt{P_c}} - \frac{L}{\sqrt{P_b}}, \qquad y_v \;=\; L\Big(\sqrt{P_c} - \sqrt{P_a}\Big).$$

Then for any trade that does not move P_c outside $[P_a, P_b]$, the V3 swap behaves as a V2 swap on (x_v, y_v) with the corresponding fee tier $\gamma_f = 1 - f/10^6$.

A self-contained derivation is beyond the scope of this paper; see [2], Section 6, and the reference implementation [3]. The operational consequence relevant here is:

Corollary 4.2 (V3 per-tier output is a ϕ-type curve)**.** *Fix a trade that remains inside a single tick range. Then the output amount is $\phi_{x_v,y_v,\gamma_f}(\delta)$, and all properties proven in Section 3 — monotonicity, boundedness, concavity — transfer verbatim to this regime.*

Remark 4.3 (On-chain V3 quoting)**.** On Ethereum, V3 quotes are obtained via the canonical `QuoterV2` contract, which simulates the swap by reverting with the computed output encoded in the revert data. Because the revert is internal to `QuoterV2`, the top-level call returns normally to the caller; our aggregator wraps each quote in a `try/catch` block so that unsupported pools (reverting outside the expected pattern) gracefully degrade to an output of 0 (`DexAggregatorV1.sol`, `getV3Quote`).

5 The Aggregator Protocol

5.1 Single-hop aggregation

Let $\mathcal{V} = \{v_1, \ldots, v_n\}$ be the venue set (Definition 2.4). For each $v_i \in \mathcal{V}$ and input $\delta \geq 0$, denote the output of venue i by $O_i(\delta) \in \mathbb{R}_{\geq 0}$; if venue i is not live then $O_i(\delta) = 0$ by convention.

Definition 5.1 (Aggregator output)**.** The *aggregator output function* is

$$O^*(\delta) \;=\; \max_{i=1,\ldots,n} O_i(\delta). \tag{4}$$

The *best venue index* at input δ is any $i^*(\delta) \in \arg\max_i O_i(\delta)$ (ties broken deterministically by the implementation; see Section 6).

Theorem 5.2 (Single-hop optimality)**.** *Let $\delta \geq 0$ and let O_i be the output functions of venues $v_1, \ldots, v_n$. Then for every $i \in \{1, \ldots, n\}$,*

$$O^*(\delta) \;\geq\; O_i(\delta).$$

The aggregator output $O^*(\delta)$ is therefore the pointwise supremum over the finite venue set $\mathcal{V}$, and is attained at $i^*(\delta)$.

Proof. Immediate from the definition of max over a finite set: for every i, $O_i(\delta) \leq \max_j O_j(\delta) = O^*(\delta)$. The maximum is attained because the index set is finite. □

Remark 5.3 (Scope of the claim)**.** Theorem 5.2 establishes optimality *within the single-hop venue set $\mathcal{V}$*. It does *not* claim optimality among all possible paths, nor optimality under dynamic strategies (order splitting, MEV-aware routing, etc.). Multi-hop and split-routing extensions can strictly improve on this bound and are the subject of the next subsection and of the V2 upgrade deployed on Sepolia.

5.2 Multi-hop extension

The V2 upgrade (`DexAggregatorV2`) extends the venue set with two-leg paths $\mathsf{In} \to m \to \mathsf{Out}$ routed through an operator-configured intermediate token m (typically WETH). We characterize when such a path strictly dominates the best direct venue.

Consider two V2 pools:

- a *direct* pool with reserves (x, y) and fee γ,
- an *intermediate pair* (In, m) with reserves (x_1, m_1) and fee γ_1, and a second pool (m, Out) with reserves (m_2, y_2) and fee γ_2.

Denote the direct and multi-hop outputs by

$$O_{\text{dir}}(\delta) = \phi_{x,y,\gamma}(\delta), \qquad O_{\text{mh}}(\delta) = \phi_{m_2,y_2,\gamma_2}\big(\phi_{x_1,m_1,\gamma_1}(\delta)\big).$$

Proposition 5.4 (Small-trade multi-hop dominance)**.** *Assume all pools are live. Then as $\delta \to 0^+$,*

$$\frac{O_{\text{mh}}(\delta)}{O_{\text{dir}}(\delta)} \;\longrightarrow\; \frac{\gamma_1\,\gamma_2}{\gamma} \cdot \frac{m_1}{x_1} \cdot \frac{y_2}{m_2} \cdot \frac{x}{y}.$$

Consequently, $O_{\text{mh}}(\delta) > O_{\text{dir}}(\delta)$ for sufficiently small $\delta > 0$ if and only if

$$\frac{\gamma_1\,\gamma_2}{\gamma} \cdot \frac{m_1}{x_1} \cdot \frac{y_2}{m_2} \;>\; \frac{y}{x}. \tag{5}$$

Proof. By Corollary 3.4, $\phi_{a,b,\gamma}(\delta) = b\gamma\delta/a + O(\delta^2)$ as $\delta \to 0$. Apply this to the inner leg:

$$\phi_{x_1,m_1,\gamma_1}(\delta) \;=\; \frac{m_1\gamma_1}{x_1}\,\delta + O(\delta^2).$$

Substituting into the outer leg and linearizing:

$$O_{\text{mh}}(\delta) \;=\; \phi_{m_2,y_2,\gamma_2}\left(\frac{m_1\gamma_1}{x_1}\delta + O(\delta^2)\right) \;=\; \frac{y_2\,\gamma_2}{m_2} \cdot \frac{m_1\,\gamma_1}{x_1}\,\delta + O(\delta^2).$$

Similarly $O_{\text{dir}}(\delta) = (y\gamma/x)\delta + O(\delta^2)$. The ratio as $\delta \to 0^+$ is therefore

$$\frac{O_{\text{mh}}(\delta)}{O_{\text{dir}}(\delta)} \;\longrightarrow\; \frac{y_2\gamma_2 m_1\gamma_1/(m_2 x_1)}{y\gamma/x} \;=\; \frac{\gamma_1\gamma_2}{\gamma} \cdot \frac{m_1}{x_1} \cdot \frac{y_2}{m_2} \cdot \frac{x}{y},$$

which is (5) after rearrangement. □

Remark 5.5 (Interpretation of (5))**.** The left-hand side is a product of three factors:

- $\gamma_1\gamma_2/\gamma \leq \gamma < 1$: the fee penalty of traversing two pools rather than one.
- m_1/x_1: the spot rate $\mathsf{In} \to m$ in the first pool.
- y_2/m_2: the spot rate $m \to \mathsf{Out}$ in the second pool.

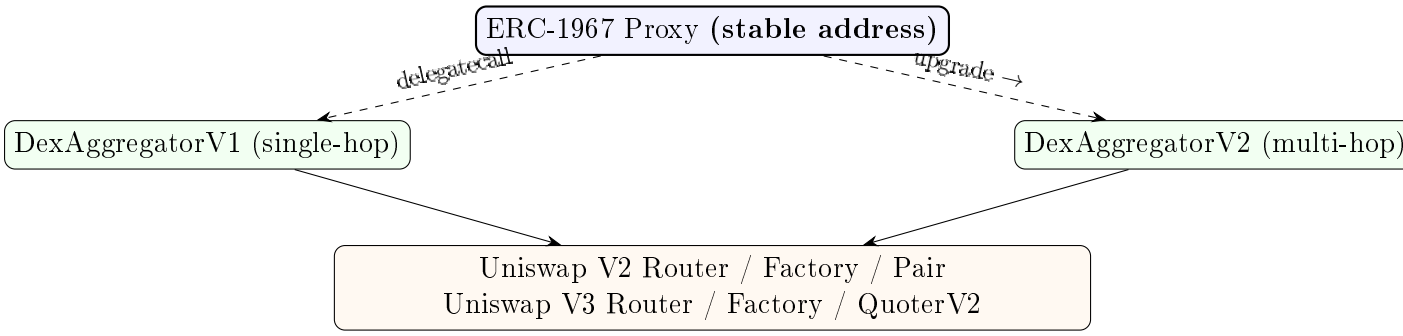
Their product is the *effective multi-hop spot rate* $\mathsf{In} \to \mathsf{Out}$ after the double fee. Multi-hop strictly dominates in the small-trade limit precisely when this effective spot rate exceeds the direct pool's spot rate y/x — typically the case when the direct pool is thinly funded relative to the two intermediate pools.

Remark 5.6 (Large-trade regime)**.** For large δ, both O_{dir} and O_{mh} saturate (Theorem 3.5), and the comparison depends on pool depths rather than spot rates. In particular, multi-hop can dominate in the large-trade regime even when (5) fails, because the direct pool saturates sooner. The aggregator resolves this by comparing outputs pointwise at execution time rather than relying on asymptotic conditions.

6 Implementation

6.1 Architecture

The system is deployed behind an ERC-1967 UUPS proxy. The proxy holds all state (storage-only) and delegates calls to an implementation contract; admins can upgrade the implementation without migrating users or liquidity, preserving the single stable address that external systems integrate against.



6.2 Storage layout (ERC-7201)

The contracts use ERC-7201 *namespaced storage* [4] to guarantee forward compatibility across upgrades. Each version declares its own struct anchored at a deterministic slot derived from the namespace string:

```
// keccak256(abi.encode(uint256(keccak256("dex-aggregator.storage.v1")) - 1))
//      & ~bytes32(uint256(0xff))
bytes32 private constant STORAGE_LOCATION_V1 =
    0xd87ff71024a4d1c4d04e88b39e8672e110e0b1a5a6ca82cf2a4710b85e0e6800;
```

Listing 1: Namespaced storage slot derivation

V2 adds its own slot (`STORAGE_LOCATION_V2`) without touching V1's layout, so upgrading is safe regardless of the order in which fields are declared.

6.3 Security model

- **Access control.** Two roles are defined in OpenZeppelin `AccessControl`: `DEFAULT_ADMIN_ROLE` (can upgrade and set the protocol fee) and `OPERATOR_ROLE` (can pause, unpause, and update the intermediate-token list in V2).
- **Reentrancy.** All state-modifying external entry points are guarded by OpenZeppelin `ReentrancyGuard`.
- **Pausable.** Swap functions are gated by `whenNotPaused`; emergency pausing is the operator's circuit breaker.
- **Deadline.** Every swap checks `block.timestamp` $\leq$ `deadline`, preventing long-pending transactions from executing at stale prices.
- **Slippage.** Every swap enforces `amountOut` $\geq$ `amountOutMin`, with `amountOutMin` supplied by the caller.
- **SafeERC20 with `forceApprove`.** Non-standard ERC20 tokens (USDT-style implementations that revert on re-approval of a non-zero allowance) are handled correctly.

6.4 Upgrade authorization

The `_authorizeUpgrade` hook is gated by `DEFAULT_ADMIN_ROLE`:

```
function _authorizeUpgrade(address) internal override
    onlyRole(DEFAULT_ADMIN_ROLE) {}
```

Combined with `_disableInitializers()` in the constructor, this ensures that only the proxy's admin can upgrade, and that the implementation contract itself cannot be initialized directly.

7 Deployment and Evaluation

7.1 Sepolia deployment

Chain id `11155111`. All contracts are verified on Sourcify [5] with exact bytecode match (creation + runtime):

Contract	Address
ERC-1967 Proxy (entry point)	`0x7b43d70CBe2658A09376eB3a8126D426dBb69501`
`DexAggregatorV1` impl.	`0x860AC17e59d2875FE24541AFAf6A8849409befdB`
`DexAggregatorV2` impl. (current)	`0x3c9544C5666FCb678eE381390a261bb8E47A6F82`
TestToken A (TKA)	`0x3F8d55D01a59e8327a4eEaf643cd8CAF0ed42993`
TestToken B (TKB)	`0x86C8107D697F99ef61019d62c985b60F11b6c58e`
Deployer / Admin	`0x11d7b17BDcaB85626660EBb0AE75a3B4827F0233`

7.2 Live swap demonstration

A representative end-to-end swap was executed on Sepolia (transaction `0xf3f831c...16eac3f11`):

1. **Off-chain quote** via `getQuote(WETH, UNI, 5e15)` returned `(DexType.V3, 142,594,212,628,689, fee=10000)`: V3 at the 1.00 % tier, predicting approximately 0.000 142,594 UNI.
2. **On-chain swap** of the same 0.005 WETH through the proxy yielded exactly 142,594,212,628,689 UNI — a bit-exact match with the off-chain quote (identical block state).

The aggregator's selection of V3 at 1.00 % out of five candidate venues (V2 and four V3 tiers) realizes Theorem 5.2 in practice.

7.3 Testing

Category	Count	Scope
Unit tests	18	Mock-based tests of V1 quoting, swap, pause, deadline, slippage, admin
Library tests	8	V2 formula correctness, path encoding, boundary conditions
Fuzz tests	4	Property-based invariants: $\phi < y$, monotonicity in δ, slippage
Fork tests	8	Live Sepolia state: real quotes, real routing, admin configuration
Total	**38**	All passing

The property-based test `testFuzz_CalcV2AmountOut_NeverExceedsReserve` exercises Theorem 3.5 directly, asserting $\phi(\delta) < y$ over 256 random inputs. The monotonicity test asserts Theorem 3.3. Together these elevate two of the mathematical guarantees from paper to machine-checked properties of the deployed code.

8 Conclusion

We have presented a DEX aggregator that quotes across Uniswap V2 and all four V3 fee tiers, routes each swap through the output-maximizing venue, and is deployed behind a UUPS proxy upgradable to multi-hop routing. The core theoretical contributions are the proofs of monotonicity, boundedness, and strict concavity of the V2 swap curve (Theorems 3.3–3.7), the formal optimality of the single-hop aggregator (Theorem 5.2), and the characterization of the small-trade regime in which multi-hop strictly dominates (Proposition 5.4). On the implementation side, the protocol enforces a standard security envelope (access control, reentrancy, pause, deadline, slippage), is verified on Sourcify, and is exercised by a 38-test suite spanning unit, property-based, and fork tests against live Sepolia liquidity.

Open directions. Natural extensions include (i) *optimal split routing*, where a single trade is divided across multiple venues to minimize price impact — this reduces to a convex optimization problem with closed-form solutions in the two-venue case; (ii) *MEV-aware routing*, integrating bundled transactions or commit-reveal schemes to reduce sandwich exposure; (iii) *cross-chain aggregation*, where venues live on different L2s and the aggregator coordinates via a bridge-aware execution layer.

References

[1] H. Adams, N. Zinsmeister, D. Robinson. *Uniswap V2 Core*. March 2020. https://uniswap.org/whitepaper.pdf

[2] H. Adams, N. Zinsmeister, M. Salem, R. Keefer, D. Robinson. *Uniswap v3 Core*. March 2021. https://uniswap.org/whitepaper-v3.pdf

[3] Uniswap Labs. *Uniswap V3 reference implementation*. https://github.com/Uniswap/v3-core

[4] F. Giordano et al. *ERC-7201: Namespaced Storage Layout*. Ethereum Improvement Proposals, 2023. https://eips.ethereum.org/EIPS/eip-7201

[5] Argot Collective. *Sourcify: decentralized source-code verification for Ethereum*. https://sourcify.dev